PAPER → CODE · ARXIV:2001.08361

Scaling Laws for Neural Language Models

Every equation of Kaplan, McCandlish et al. (OpenAI, 2020), extracted and explained parameter by parameter — with the fitted constants and a live-computed plot for each law. Test loss L is the cross-entropy in nats/token on WebText2.

The math explained — companion to the interactive site

Original authors: J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, et al.

Visualised & annotated by **Abdulsamad Teniola Muyideen**

13 core equations · Tables 4–6 constants · 6 figures

§3 · EMPIRICAL RESULTS

The three basic power laws

L(N) — PARAMETER LAW

$$L(N) = \left(\frac{N_c}{N}\right)^{\alpha_N} \tag{1.1}$$

What it does. Predicts the converged test loss of a model with N non-embedding parameters when the dataset is large enough that data is not the bottleneck. On a log-log plot this is a straight line: multiplying N by any factor drops the loss by a fixed factor. Doubling N multiplies the loss by 2^-0.076 ≈ 0.95 (a 5% reduction).

SYMBOL	MEANING	VALUE
N	number of non-embedding parameters (excludes token & positional embeddings)	–
α_N	parameter power-law exponent	0.076
N_c	parameter scale — tokenizer-dependent, no fundamental meaning	8.8×10^{13}

L(D) — DATA LAW

$$L(D) = \left(\frac{D_c}{D}\right)^{\alpha_D} \tag{1.2}$$

What it does. The loss of a large model trained with early stopping on a limited dataset of D tokens — now data is the bottleneck. Same functional shape as L(N) but a steeper slope, so each doubling of data buys slightly more than each doubling of parameters.

SYMBOL	MEANING	VALUE
D	dataset size in tokens	–
α_D	data power-law exponent	0.095
D_c	data scale (tokens)	5.4×10^{13}

L(C_MIN) — COMPUTE LAW

$$L(C_{\min}) = \left(\frac{C_c^{\min}}{C_{\min}}\right)^{\alpha_C^{\min}} \tag{1.3}$$

What it does. The best loss reachable for a given budget of optimally-allocated compute: a sufficiently large dataset, an optimally-sized model, and a batch size well below B_crit. This is the trend to use for real compute forecasts (the naive fixed-batch trend L(C) is messier).

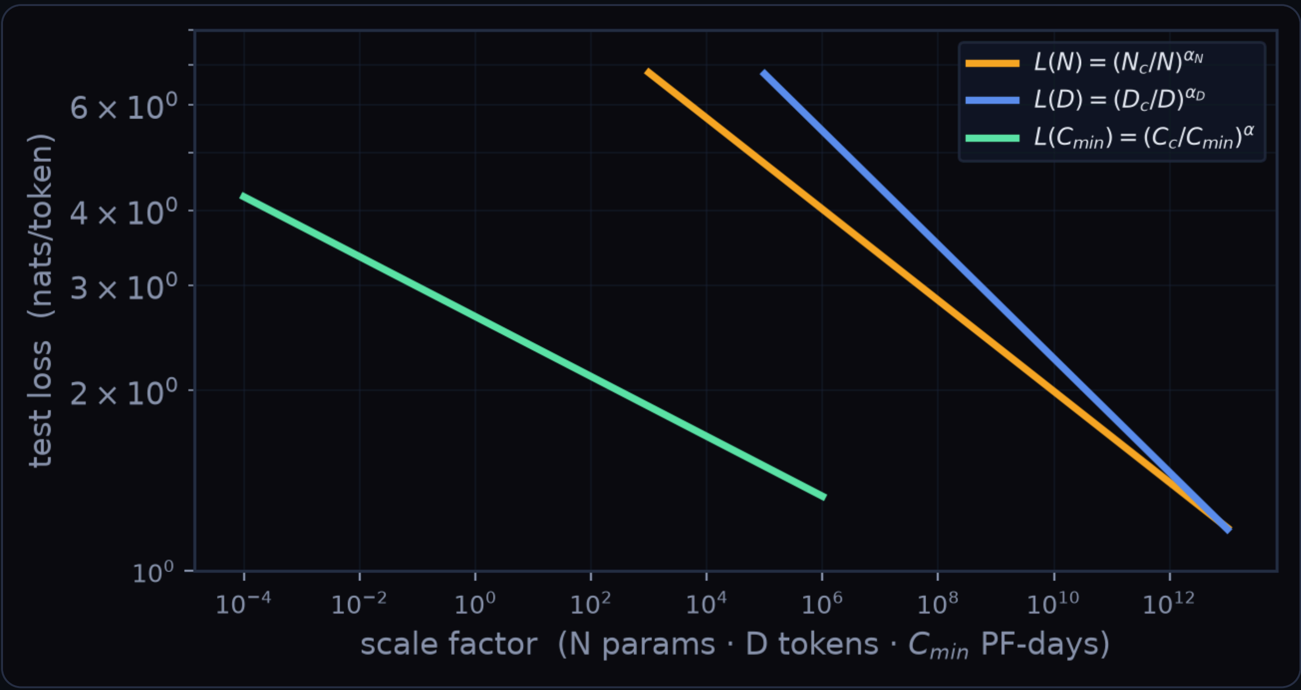
SYMBOL	MEANING	VALUE
C_min	optimal non-embedding compute (PF-days; 1 PF-day = 8.64×10^{19} FLOP)	–
α_C^min	optimal-compute exponent	0.050
C_c^min	compute scale (PF-days)	3.1×10^{8}

L(C) — NAIVE FIXED-BATCH COMPUTE LAW (3.3)

$$L(C) = \left(\frac{C_c}{C}\right)^{\alpha_C}$$

What it does. The empirical loss versus raw training compute C when the batch size is held fixed. Slightly steeper and lumpier than the L(C_min) trend; retained only for comparison.

SYMBOL	MEANING	VALUE
C	raw training compute (PF-days), C = 6·N·B·S	—
α_C	naive compute exponent	0.057
C_c	naive compute scale (PF-days)	1.6×10^{7}



L(N) amber · L(D) blue · L(C_min) green — each a straight line over 6–8 orders of magnitude.

Compute & parameter counting

NON-EMBEDDING PARAMETER COUNT

(2.1)

$$N \approx 2\, d_{model}\, n_{layer}\, (2\, d_{attn} + d_{ff}) = 12\, n_{layer}\, d_{model}^2$$

What it does. How the model size N is computed from the architecture, using the standard proportions d_attn = d_ff/4 = d_model. Embedding and positional parameters are deliberately excluded — doing so is what makes the scaling laws clean.

SYMBOL	MEANING	VALUE
n_layer	number of Transformer layers	–
d_model	residual-stream width	–
d_ff, d_attn	feed-forward and attention widths	d_ff=4·d_model

COMPUTE PER TOKEN

(2.2)

$$C_{forward} \approx 2N + 2\, n_{layer}\, n_{ctx}\, d_{model}\ ; \qquad C \approx 6N \text{ per token}$$

What it does. Forward-pass FLOP per token; the factor of 2 is the multiply-accumulate. Adding the backward pass (≈2× forward) gives the rule of thumb C ≈ 6N FLOP per token, which underlies C = 6·N·B·S for a whole run.

SYMBOL	MEANING	VALUE
n_ctx	context length in tokens	1024
B	batch size (tokens)	–
S	number of parameter updates (steps)	–

Critical batch size

B_CRIT(L) — CRITICAL BATCH SIZE (1.4)

$$B_{\text{crit}}(L) = \frac{B_*}{L^{1/\alpha_B}}$$

What it does. The batch size giving a roughly optimal time/compute trade-off. Batches up to B_crit cost almost no extra compute; beyond it you get diminishing returns. It depends only on the current loss L — not on model size — and roughly doubles for every 13% drop in loss, so as training improves you can parallelise harder.

SYMBOL	MEANING	VALUE
L	current loss (nats/token)	–
B_*	batch scale	2.1×10^8 tok
α_B	batch exponent (1/α_B ≈ 4.8)	0.21

STANDARDISED STEPS S_MIN (5.4)

$$S_{\text{min}} = \frac{S}{1+B_{\text{crit}}(L)/B}$$

What it does. Converts the actual number of steps S taken at batch size B into the minimum steps needed at B ≫ B_crit. This universal S_min is what the learning-curve fits are parameterised in.

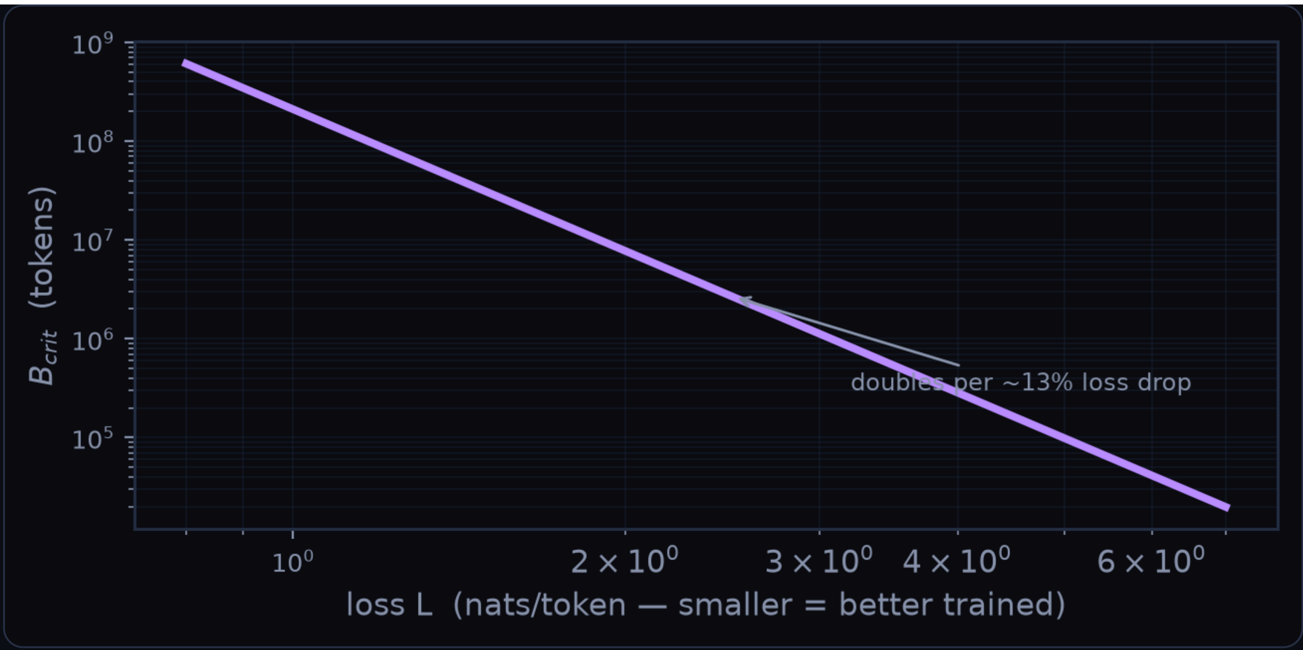
SYMBOL	MEANING	VALUE
S	steps actually taken at batch B	–
B	the batch size used	–

STANDARDISED COMPUTE C_MIN (5.5)

$$C_{\text{min}} = \frac{C}{1+B/B_{\text{crit}}(L)}$$

What it does. Converts raw compute C = 6·N·B·S into the minimum compute you would have spent at B ≪ B_crit — the clean quantity used for all forecasts.

SYMBOL	MEANING	VALUE
C	raw compute spent (PF-days)	–



B_crit shoots up as the loss falls, diverging as L→0 near the entropy floor.

Model size × data — overfitting

L(N,D) — THE UNIFIED LAW

(1.5)

$$L(N, D) = \left[\left(\frac{N_c}{N}\right)^{\alpha_N/\alpha_D} + \frac{D_c}{D}\right]^{\alpha_D}$$

What it does. A single equation for both knobs at once. As D→∞ it collapses to L(N); as N→∞ it collapses to L(D). In between, whichever resource is scarcer dominates. Built so a change of vocabulary merely rescales N_c and D_c.

SYMBOL	MEANING	VALUE
N	params	–
D	tokens	–
α_N/α_D	coupling ratio	≈ 0.80

ΔL — THE OVERFITTING PENALTY

(4.3)

$$\delta L(N, D) = \frac{L(N,D)}{L(N,\infty)} - 1 = \left[1 + \left(\frac{N}{N_c}\right)^{\alpha_N/\alpha_D} \frac{D_c}{D}\right]^{\alpha_D} - 1$$

What it does. The fractional extra loss caused by a finite dataset, relative to the infinite-data floor L(N). Remarkably it depends only on the single combination ~ N^0.74/D, so every model size collapses onto one universal curve.

SYMBOL	MEANING	VALUE
$L(N, \infty)$	= L(N), the infinite-data floor	–

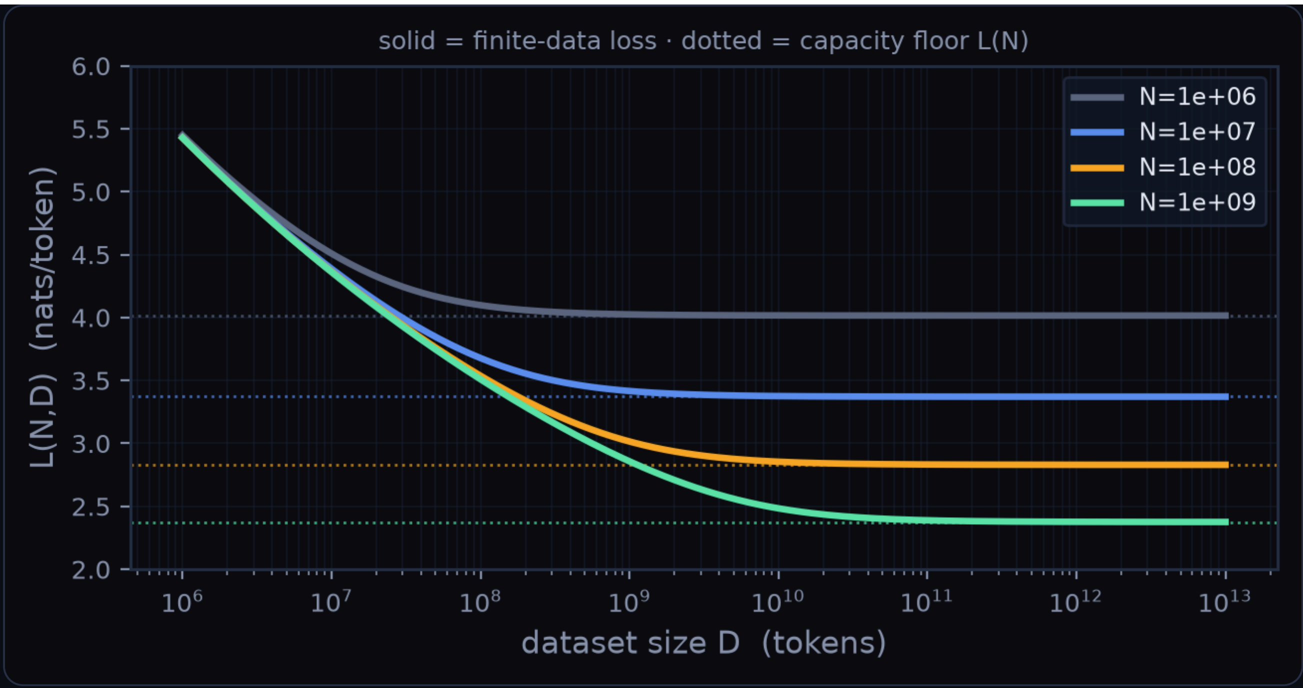
NO-OVERFIT DATA REQUIREMENT

(4.4)

$$D \gtrsim (5 \times 10^3)\, N^{0.74}$$

What it does. The minimum dataset size that keeps overfitting under the ~0.02 random-seed noise floor. Data need only grow sub-linearly in model size: grow N by 8× and D by just ~5×. The exponent 0.74 = 1 − α_N/α_D.

SYMBOL	MEANING	VALUE
0.74	= 1 − α_N/α_D	–



Solid = finite-data L(N,D); dotted = capacity floor L(N). Where they meet, more data stops helping.

Model size × training time

L(N,S_MIN) — LEARNING-CURVE LAW

(1.6)

$$L(N, S_{\min}) = \left(\frac{N_c}{N}\right)^{\alpha_N} + \left(\frac{S_c}{S_{\min}}\right)^{\alpha_S}$$

What it does. Loss as a capacity floor (first term, set by N) plus a training-progress term (second term, falling with steps). Because the shape is nearly independent of N, you can fit the early part of a run and extrapolate where it will end up.

SYMBOL	MEANING	VALUE
S_min	steps measured at B ≫ B_crit	–
α_S	step exponent	0.76
S_c	step scale	2.1×10³

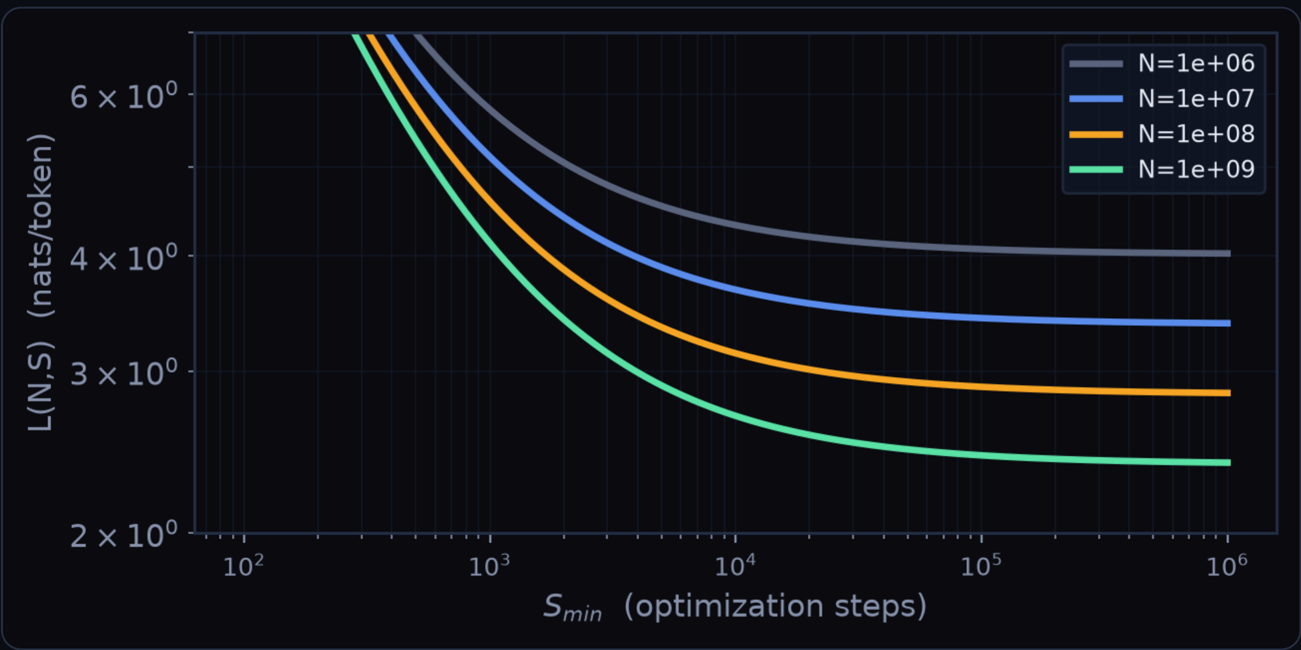
EARLY-STOPPING LOWER BOUND

(5.7)

$$S_{\text{stop}}(N, D) \gtrsim \frac{S_c}{\left[L(N,D) - L(N,\infty)\right]^{1/\alpha_S}}$$

What it does. A lower-bound estimate of the step at which to stop when training is data-limited: the larger the finite-data gap over the infinite-data floor, the sooner you should stop.

SYMBOL	MEANING	VALUE
$L(N,D) - L(N,\infty)$	the overfitting gap	–



Bigger models (brighter) start and finish lower, and reach any target loss in fewer steps.

Optimal allocation of compute

COMPUTE-EFFICIENT FRONTIER (1.7)

$$N_{\text{opt}} \propto C_{\min}^{0.73}, \quad B \propto C_{\min}^{0.24}, \quad S_{\min} \propto C_{\min}^{0.03}, \quad D_{\text{opt}} \propto C_{\min}^{0.27}$$

What it does. How to spend a budget. Model size rockets up (exponent 0.73) while serial steps barely move (0.03). A billion-fold more compute should become an ~billion-fold bigger model with almost no extra training time — the paper's central practical message.

SYMBOL	MEANING	VALUE
N_e	model scale	1.3×10^{9}
B_e	batch scale	2.0×10^{6}
S_e	step scale	5.4×10^{3}
D_e	data scale	2.0×10^{10}

THE EXPONENT, PREDICTED (6.4)

$$\alpha_C^{\min} = \frac{1}{1/\alpha_S + 1/\alpha_B + 1/\alpha_N} \approx 0.054$$

What it does. The compute exponent is not free — it is derived from the N, S and B exponents. The prediction (0.054) matches the direct fit (0.050), a genuine consistency check that the whole framework hangs together.

SYMBOL	MEANING	VALUE
$\alpha_N, \alpha_S, \alpha_B$	the three base exponents	0.076 / 0.76 / 0.21

N (amber) climbs far faster than D (blue); S (green) is nearly flat.

Where the laws must break

SINGLE-EPOCH DATA USAGE (6.7)

$$D(C_{\min}) = \frac{2\,C_{\min}}{6\,N(C_{\min})} \approx (4\times10^{10})\,C_{\min}^{0.26}\ \text{tokens}$$

What it does. The most data compute-efficient training can consume without reusing any (one epoch). It grows so slowly that a big model eventually starves for fresh data relative to its size.

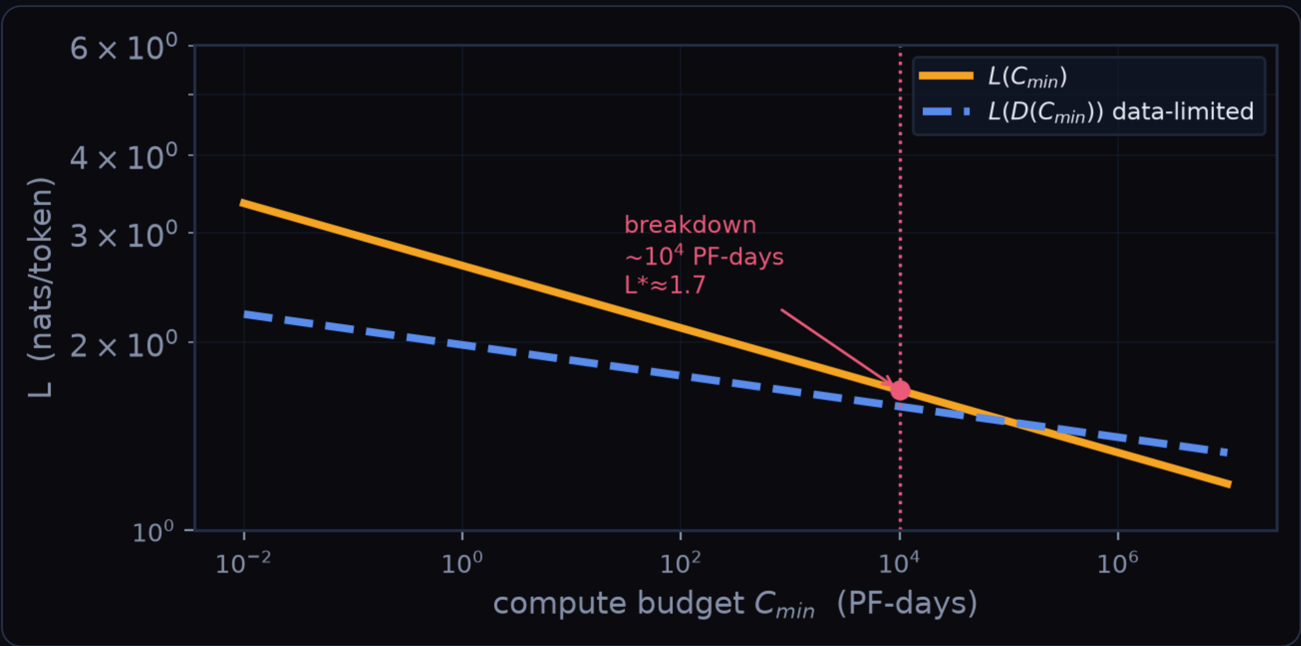
SYMBOL	MEANING	VALUE
$2C_min$	train at critical batch, C = 2·C_min	–

THE BREAKDOWN POINT (6.8)

$$C^* \sim 10^4\ \text{PF-days},\quad N^* \sim 10^{12},\quad D^* \sim 10^{12},\quad L^* \sim 1.7\ \text{nats}$$

What it does. Where the compute law L(C_min) dips below the data law L(D) — a contradiction. The paper conjectures this crossing is roughly where Transformers reach maximal performance and the power laws must bend. The numbers are uncertain by an order of magnitude.

SYMBOL	MEANING	VALUE
$L*$	conjectured entropy-per-token estimate	≈ 1.7 nats



L(C_min) amber vs. the data-limited floor L(D(C_min)) blue — they cross near 10⁴ PF-days.